

EJERCICIO PRÁCTICO N° 2.1

Objetivos

- Realizar la instalación, despliegue y parametrización de servidores web de uso en la industria (Apache y Nginx) bajo entornos Linux.
- Comprender y aplicar la configuración de VirtualHosts (Apache) y Server Blocks (Nginx) para gestionar puertos de escucha, asignación de directorios, permisos y alojamiento de múltiples sitios web.
- Parametrizar puertos de escucha en servidores web para gestionar la publicación de sitios, permitiendo el despliegue de entornos alternativos y la resolución de conflictos entre servicios.
- Interpretar y analizar archivos de logs (registros de acceso y errores) para monitorear el tráfico, el comportamiento de las peticiones HTTP y diagnosticar el funcionamiento del servidor.

Requerimientos:

- Cliente Putty (acceso SSH).
- Máquina virtual Linux en versión Ubuntu 22.04. Requerimientos mínimos recomendados: 1Gb RAM, 1CPU, 20Gb disco, 1 interfaz Red Bridge.

Ejercicio 1:

1. Acceder al servidor a través de SSH para su configuración desde una terminal SSH.
2. Instalar un servidor apache en la distribución de Linux utilizada. Indique los comandos utilizados e indique si el mismo lo instaló en un host físico o VM.
3. Crear virtual host y parametrización para alojar diferentes sitios web.
4. Localizar e interpretar los diferentes tipos de logs disponibles en el servidor.
5. Realizar parametrización y definición de permisos a través de .htaccess.
6. Realizar las configuraciones necesarias para alojar en el servidor web una página HTML en el puerto 80. Puede utilizar algún sitio que tenga creado previamente.
7. Configurar el servidor para publicar en el puerto 80 en el directorio "/admin" la siguiente página HTML. Reemplazar [XXXXXX] por la LU del alumno.

```
<!doctype html>
<html lang="es">
<head>
  <meta charset="utf-8">
  <title>Redes de Datos </title>
  <style>
    body {
      font-family: system-ui, sans-serif;
      background: #f1f5f9;
```

```
        display: grid;
        place-items: center;
        min-height: 100vh;
        margin: 0;
    }
    #container {
        background: #fff;
        padding: 2rem;
        border-radius: 12px;
        box-shadow: 0 4px 12px rgba(0,0,0,0.1);
        width: 280px;
    }
    h1 { font-size: 1.4rem; text-align: center;
margin-top: 0; color: #0f172a; }
    label { font-size: 0.85rem; font-weight: 600;
display: block; margin: 0.8rem 0 0.2rem; }
    input { width: 100%; padding: 0.6rem; border:
1px solid #cbd5e1; border-radius: 6px; box-sizing:
border-box; }
    input[type="submit"] { background: #2563eb;
color: #fff; border: none; font-weight: 600; margin-top:
1.2rem; cursor: pointer; }
    input[type="submit"]:hover { background:
#1d4ed8; }
</style>
</head>
<body>
    <div id="container">
        <h1>Redes de Datos | LU: [XXXXXX]</h1>
        <form>
            <label for="user">Usuario:</label>
            <input type="text" id="user" required>
            <label for="pass">Contraseña:</label>
            <input type="password" id="pass" required>
            <input type="submit" value="Login">
        </form>
    </div>
</body>
</html>
```

Ejercicio 2:

1. Se requiere publicar una nueva página web en el mismo servidor en el puerto 8080.
2. Realizar las configuraciones necesarias para publicar el mismo.

El contenido del HTML a alojar en esta página es:

```
<!doctype html>
<html>
<head>
  <meta charset="utf-8">
  <meta http-equiv="X-UA-Compatible"
content="IE=edge,chrome=1">
  <!-- Title Page -->
  <title> Mi Sitio WEB - Puerto 8080</title>
</head>
<body>
  <div id="container">
    <H1> Nuevo sitio de Redes de Datos - Acceso a
través del puerto 8080</H1>
  </div>
</body>
</html>
```

3. Indicar cuales son las modificaciones necesarias, adjuntar captura de pantalla del navegador de la PC accediendo a la página web configurada.

Ejercicio 3:

1. Usar curl para visualizar headers y cuerpo. Ejecutar en la máquina host:

```
curl -i http://<IP_DEL_SERVIDOR>
```

Identificar y explicar.

- Status Line
- Date
- Server
- Content-Length
- Content-Type

¿Qué diferencia se tiene con la información mostrada por el navegador WEB?

Luego ejecutar lo siguiente, y explicar la respuesta obtenida:

```
curl -I http://<IP_DEL_SERVIDOR>/noexiste.html
```

2. Visualizar los logs del servidor, indicar con una captura de pantalla los resultados obtenidos.

Tomar una entrada de registro e identificar:

- IP cliente,
- Método HTTP
- Recurso solicitado
- Código HTTP

Ejercicio 4:

1. Instalar el servidor Nginx en su máquina virtual ubuntu. Indique el comando utilizado para la instalación. (Se puede utilizar una estado anterior de la VM, previo a la instalación de apache).
2. Parametrización: Crear un nuevo bloque de servidor (Server Block) en Nginx dentro del directorio /etc/nginx/sites-available/ y habilitarlo creando su enlace simbólico correspondiente en sites-enabled/. El directorio se debe llamar: wwwredesnginx

Configurar el servidor para publicar en el **puerto 80** un HTML index.html que contenga el siguiente código HTML:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Redes de Datos</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      background-color: #f4f4f9;
      color: #333;
      text-align: center;
      margin-top: 100px;
    }
    h1 {
      color: #802704;
      font-size: 3em;
    }
    h2 {
      color: #444;
      font-size: 2em;
    }
  </style>
```

```
</head>
<body>
  <h1>Bienvenido a Nginx - Redes de Datos | LU:
[XXXXXX]</h1>
  <h2>HTTP con Nginx</h2>
</body>
</html>
```

3. Modificar el HTML anterior que se publicó en el puerto 80 para agregar un script que realice una petición HTTP GET a una API pública gratuita. Por ejemplo ipify la cual devuelve la dirección IP del cliente WEB en formato JSON.

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Redes de Datos</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      background-color: #f4f4f9;
      color: #333;
      text-align: center;
      margin-top: 100px;
    }
    h1 {
      color: #802704;
      font-size: 3em;
    }
    h2 {
      color: #444;
      font-size: 2em;
    }
  </style>
</head>
<body>
  <h1>Redes de datos - TP</h1>
  <h2>HTTP con Nginx</h2>

  <!-- Contenedor para el resultado de la API -->
```

```
<div id="api-container" style="margin-top: 2rem;
padding: 1rem; background: #e2e8f0; border-radius: 8px;
text-align: center; display: inline-block;">
  <strong>Consulta a API Externa:</strong>
  <p id="api-result">Realizando petición
HTTP...</p>
</div>
```

```
<!-- Script para consumir la API -->
<script>
  // Petición HTTP GET a una API pública
  fetch('https://api.ipify.org?format=json')
    .then(response => {
      if (!response.ok) throw new Error('Error
en la respuesta HTTP');
      return response.json();
    })
    .then(data => {

document.getElementById('api-result').innerText = 'La IP
Pública del cliente es: ' + data.ip;

    })
    .catch(error => {

document.getElementById('api-result').innerText = 'Fallo
en la conexión a la API.';
      console.error(error);
    });
</script>
</body>
</html>
```

Entrega: El alumno debe entregar un documento que indique las configuraciones y verificaciones realizadas. Ambos ejercicios deben tener la nomenclatura: "TP2.1_LUApellidoNombre".

Rúbricas evaluativas

Criterio	Excelente (100%)	Bueno (75%)	A mejorar (50%)	Insuficiente (0-25%)
1. Acceso y configuración inicial	Acceso vía SSH exitoso. Apache instalado correctamente en VM o físico. Comandos documentados paso a paso.	Acceso y Apache configurados con éxito, con 1–2 errores menores en comandos o documentación.	Configuración parcial, con errores que dificultan el funcionamiento.	No se logró acceso SSH o no se instaló Apache.
2. Configuración de Virtual Hosts y sitios	VirtualHost configurado correctamente, alojando varios sitios. Directorio /admin creado con la página solicitada funcionando.	VirtualHost configurado con parámetros correctos en general, errores menores (nombres, rutas). Página en /admin accesible.	Configuración incompleta, página en /admin no accesible o mal parametrizada.	No se configuró VirtualHost o no funciona.
3. Manejo e interpretación de logs	Identificación clara de logs de Apache (error, access, etc.), explicación correcta de una entrada y fundamentación de su importancia.	Logs localizados y explicados en general, aunque con detalles incompletos.	Logs localizados pero explicación confusa o incompleta, sin justificar utilidad.	No se presentan logs ni explicación.
4. Configuración de servicio en puerto 8080	Sitio adicional publicado en el puerto 8080, accesible desde navegador con captura de pantalla.	Sitio en 8080 configurado y accesible, con detalles menores en la configuración.	Sitio en 8080 configurado parcialmente o con fallas de acceso.	No se configuró el sitio en puerto 8080 o no responde.
5. Implementación de Nginx	Servidor Nginx instalado y parametrizado correctamente mediante un Server Block. Consumo de API externa correcto.	Nginx está configurado y el sitio es accesible en el puerto 80. El script de consumo de la API funciona, pero existen omisiones en la explicación o las capturas son incompletas.	Configuración parcial de Nginx (ej. el sitio es accesible pero persisten errores de configuración). El HTML se visualiza pero el script de la API falla.	No se logró instalar o iniciar Nginx, no se configuró el Server Block requerido y no hay evidencia de la integración de la página HTML ni del script de consumo de API.
6. Documentación	Documento estructurado, capturas de pantalla nítidas, comandos documentados, explicaciones detalladas. Archivo en formato y nomenclatura correctos.	Documento organizado en general, con leves omisiones de formato o secciones poco claras. Capturas y comandos presentes pero no siempre explicados.	Documento incompleto o desordenado, capturas poco claras, sin consistencia en formato o nomenclatura.	Documento ausente, muy incompleto o sin evidencias de actividades realizadas.